Round 4 — adversarially review the clone concurrency fix

You have GitHub read access to this repo (`tom2025b/git-vista`, public). Fetch the files named below at their real paths on `main` yourself — do not ask to have them pasted, and do not guess at their contents.

Before answering anything, confirm you can actually read the source: quote the exact string `refuse_if_lan_view` returns on its error path (`crates/git-vista/src/api.rs`). If you cannot find it, say so and stop — a prior round produced a plausible-sounding but worthless answer built from a prompt alone after its source files failed to upload.

Files to fetch

- `crates/git-vista-server/src/handlers/clone.rs` — the whole file. This is what changed across three rounds today: a client/server timeout pair, a `Drop`-guard cleanup extracted into `run_guarded`, and (most recent) an in-progress guard (`CLONES_IN_PROGRESS`, `claim_in_progress`, `InProgressGuard`) that refuses a second overlapping request under the same idempotency key.
- `crates/git-vista/src/api.rs` — the frontend HTTP client. Look at `with_deadline`, `send_write_with_key`, `send_read`, `REQUEST_TIMEOUT_MS` (60s), and `CLONE_TIMEOUT_MS` (570s, clone's own longer bound).
- `crates/git-vista-server/src/operations.rs` — read the module doc and `admit()`. This is the pattern the in-progress guard mirrors for tracked writes generally; understanding it is what lets you judge whether clone's narrower version is actually equivalent or is missing something `admit()` has.

What has already been found and fixed — do not re-derive

Two earlier rounds against this same code found real things:

1. **The retry-after-timeout double-execution window.** The client does not abort its first attempt (no `AbortController`), so a second request under the same key can genuinely arrive while the first is still executing server-side. **This is now closed** by the in-progress guard in `clone.rs` — verified with a forced- overlap test (`overlapping_clone_attempts_for_the_same_key_are_not_both_admitted`) and a paired-negative (the guard disabled, the test shown to fail, then restored).
2. **Cleanup living in a timeout match-arm skips on cancellation.** Measured against a standalone axum server: client disconnect drops the handler future and skips every match arm, not just the timeout's. Cleanup moved to a `Drop` guard (`run_guarded`'s internal `DestGuard`), which cancellation cannot skip. `kill_on_drop(true)` was also added so a cancelled handler can't orphan a running `git clone` child. Two tests exist for this: `guarded_timeout_removes_the_destination` and its paired positive `guarded_success_keeps_the_destination` (proving cleanup doesn't fire on every outcome, only on failure/timeout).

What is genuinely still open — attack these

1. **Is the in-progress guard's scope actually equivalent to `admit()`'s, or narrower in a way that matters?** `admit()` (in `operations.rs`) does more than admit-or-reject — read its doc comment on what a losing concurrent caller experiences. Does clone's guard match that, or does a second request just get a flat refusal where a tracked operation would get something better (e.g. told to await/poll the winner)? Is that difference acceptable here, and say why or why not.
2. **The `HashSet<IdempotencyKey>` guard has no TTL / no eviction path visible without reading further.** If a request panics or the process is killed between claiming the key and the `Drop` guard running, could a key be stuck claimed forever, permanently refusing all future clones under that key? Trace this by hand — does `InProgressGuard`'s `Drop` genuinely run on every exit path including a panic unwind, or only on normal return?
3. **The client/server timeout asymmetry.** Client's `CLONE_TIMEOUT_MS` is 570s; server's own bound (read the constant in `clone.rs`) is 600s. Confirm the client number really is comfortably under the server's, and that nothing about the in-progress guard changes the reasoning for why that margin needs to exist.
4. **The timer leak question from round 1, never actually measured.** `with_deadline` in `api.rs` fires a `leptos::set_timeout` on every attempt including the ones that win the race — reasoned as harmless, never measured. If you can reason about WASM/browser timer handle lifecycle more precisely than "probably fine," do so; otherwise say plainly this remains unmeasured.

Answer format

```
## Verdict
Sound / flawed / flawed-but-close, one paragraph.

## Defects found
Numbered, each with the exact triggering sequence and severity. "None" is a
valid answer if you genuinely could not break it — say which attack you tried
hardest.

## Could not determine
Name the exact file/function you'd need next, if any.

## Recommended changes
Concrete and minimal.
```